

Practical 3

About this unit

Practical 3

Practice Lab Assignment

Unit • 100% completed

Lab Assignment

Unit • 100% completed

3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c , representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

numpyarr...

```
1 import numpy as np
2
3 # write your code here...
4 rows, cols=map(int,input().split())
5 elements=[]
6 for _ in range(rows):
7     elements.extend(map(int,input().split()))
8 array=np.array(elements).reshape(rows,cols)
9 print(array)
10 print(array.ndim)
11 print(array.shape)
12 print(array.size)
```

Average time

0.012 s

12.00 ms

Maximum time

0.021 s

21.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1 21 ms

Debug

Expected output

```
3 4
1 2 3 4
5 6 7 8
9 10 11 12
[[1 2 3 4]]
```

Actual output

```
3 4
1 2 3 4
5 6 7 8
9 10 11 12
[[1 2 3 4]]
```

Terminal

Test cases

Prev

Reset

Submit

Next

3.2.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a · matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

matrixOp...

1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 sum=matrix_a+matrix_b
13 print("Addition (A + B):")
14 print(sum)
15

Average time
0.024 s
24.25 ms

Maximum time
0.028 s
28.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 28 ms Debug
Expected output
Enter Matrix A:
1 2 3
Actual output
Enter Matrix A:
1 2 3

Terminal Test cases

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

11:49

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

+

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 print("Horizontal Stack:")
12 horizontal_stack=np.hstack((arr1,arr2))
13 print(horizontal_stack)
14 # Perform vertical stacking (vstack)
15 print("Vertical Stack:")
16 vertical_stack=np.vstack((arr1,arr2))
17 print(vertical_stack)
```

Average time

0.024 s

24.25 ms

Maximum time

0.028 s

28.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 28 ms

Debug

Expected output

Actual output

Terminal

Test cases

< Prev Reset Submit Next >

3.2.3. Numpy: Custom Sequence Generation 04:54

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases +

customS...

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9 sequence=np.arange(start,stop,step)
10 # Print the generated sequence
11 print(sequence)
```

Average time
0.014 s
14.25 ms

Maximum time
0.018 s
18.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 18 ms Debug

Expected output	Actual output
3	3
10	10
2	2
[3 5 7 9]	[3 5 7 9]

Terminal

Test cases

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitw... 11:57

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

- 1. Arithmetic Operations:**
 - Compute the element-wise sum, difference, and product of the two arrays.
- 2. Statistical Operations:**
 - Calculate the mean, median, and standard deviation of array A.
- 3. Bitwise Operations:**
 - Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_1 \text{ OR } B_1$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

different...

3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

def array_operations(A, B):
 # Convert A and B to NumPy arrays
 A=np.array(A)
 B=np.array(B)
 # Arithmetic Operations
 sum_result = A+B
 diff_result = A-B
 prod_result = A*B

 # Statistical Operations
 mean_A = np.mean(A)
 median_A = np.median(A)
 std_dev_A = np.std(A)

 # Bitwise Operations
 and_result = A & B
 or_result = A | B
 xor_result = A ^ B

Average time
0.020 s
20.00 ms

Maximum time
0.028 s
28.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Terminal Test cases

3.2.5. Numpy: Copying and Viewing Arrays

05:36



The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: `<original_array>`

View array: `<view_array>`

- After modifying the copy:

Original array after modifying copy: `<original_array>`

Copy array: `<copy_array>`

Sample Test Cases



Explorer

copyAnd...

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array =original_array.view()
10
11 # Create a copy
12 copy_array =original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_
17 print("View array:", view_array)
```

Average time

0.007 s

7.25 ms

Maximum time

0.010 s

10.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1

10 ms

Debug



Expected output

Actual output

Terminal

Test cases

< Prev

3.2.5. Numpy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: <original_array>
View array: <view_array>

- After modifying the copy:

Original array after modifying copy: <original_array>
Copy array: <copy_array>

Sample Test Cases

copyAnd...

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array =original_array.view()
10
11 # Create a copy
12 copy_array =original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
```

Average time
0.007 s
7.25 ms

Maximum time
0.010 s
10.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 10 ms Debug

Expected output

Actual output

Terminal Test cases

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

07:47

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching:** Find the indices where search_value appears in array1 and print these indices.
2. **Counting:** Count how many times count_value appears in array1 and print the count.
3. **Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. **Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split()))))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 A=np.where(array1==search_value)[0]
13 print(A)
14 # Count occurrences in array1
15 B=np.count_nonzero(array1==count_value)
16 print(B)
17 # Broadcasting addition
18 C= array1+broadcast_value
19 print(C)
20 # Sort the first array
```

Average time

0.022 s

22.50 ms

Maximum time

0.029 s

29.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of

Operation...

Submit

Debugger

1 import numpy as np

2

3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

4 # 1. Print all student details

5 print("All student Details:\n", a)

6

7 # 2. Print total students

8 print("Total Students:", len(a))

9

10 # 3. Print all student Roll numbers

11 print("All Student Roll Nos", a[:, 0])

12

13 # 4. Print subject 1 marks

14 print("Subject 1 Marks", a[:, 1])

15

16 # 5. Print minimum marks of Subject 2

17 print("Min marks in Subject 2", np.min(a[:, 2]))

18

19 # 6. Print maximum marks of Subject 3

20 print("Max marks in Subject 3", np.max(a[:, 3]))

21

Average time 0.025 s 25.00 ms

Maximum time 0.025 s 25.00 ms

1 out of 1 shown test case(s) passed

Test case 1 25 ms Debug

Terminal Test cases